

Systemy operacyjne

Lista zadań nr 0

Na zajęcia 6–8 października 2026

Należy przygotować się do zajęć czytając następujące rozdziały książek:

- Operating Systems: Three Easy Pieces – **Introduction**¹
- Computer Systems: A Programmer's Perspective (3rd edition) – 7.4, 7.8, 7.9, 8.1, 9.7

UWAGA! W trakcie prezentacji należy zdefiniować pojęcia oznaczone **wytluszczoną** czcionką.

Rozwiązania zadań należy prezentować przy pomocy rzutnika lub telewizora (w zależności od sali). Każde rozwiązanie należy starannie przygotować do prezentacji przed zajęciami – najlepiej w postaci pliku tekstowego z listą poleceń do wykonania i komentarzami.

UWAGA! Każdy student **musi** przygotować skrypt (lub inne gotowe polecenie), który przy pomocy programu **xrandr**² (na X11) – lub programu specyficznego dla używanego kompozytora zgodnego z protokołem Wayland – ustawi rozdzielczość ekranu wbudowanego na 1920×1080 i sklonuje go na zewnętrzne złącze VGA lub HDMI.

UWAGA 2! Każdy student **musi** przygotować skrypt (lub inne gotowe polecenie bądź konfigurację), który szybko i sprawnie dla używanego programu terminala ustawi dużą czcionkę (około 32 wierszy w trybie pełnoekranowym), kontrastowe kolory i jasne tło.

UWAGA 3! Studenci z laptopami mającymi złącza inne niż VGA/HDMI muszą mieć podczas ćwiczeń działającą przejściówkę do jednego z wymienionych złączy, w przypadku braku, prowadzący – począwszy od listy 1 – może skreślić zadanie i postawić jeden punkt ujemny.^{3 4}

Zadanie 1 (0 punktów). Przed, w trakcie, bądź po zajęciach, przetestuj, czy Twoja konfiguracja laptopa umożliwia szybkie i sprawne zaprezentowanie zadania zgodnie z uwagami z początku tej listy. Od następnej listy, za zbędne przeciąganie ze względu na problemy techniczne, prowadzący będzie miał prawo skreślić zadanie i wystawić punkty ujemne.

Zadanie 2. Czym jest **wyjątek procesora** (ang. *exception*)? Opisz różnice między **przerwaniem sprzętowym** (ang. *hardware interrupt*), **wyjątkiem programowym** (ang. *fault*) i **pułapką** (ang. *trap*).⁵ Dla każdego z nich podaj co najmniej trzy przykłady zdarzeń, które je wyzwalają. W jakim scenariuszu wyjątek procesora nie oznacza błędu czasu wykonania programu? Kiedy pułapka jest generowana w wyniku prawidłowej pracy programu?

Zadanie 3. Opisz mechanizm **obsługi przerw** bazujący na **wektorze przerwań** (ang. *interrupt vector table*). Co robi procesor przed pobraniem pierwszej instrukcji **procedury obsługi przerwania** (ang. *interrupt handler*) i po natrafieniu na instrukcję powrotu z przerwania? Cemu procedura obsługi przerwania powinna być wykonywana w **trybie jądra** (ang. *kernel mode*) i używać stosu odrębnego od stosu użytkownika?

Zadanie 4 (bonusowe). Na podstawie **entry.S**⁶ opowiedz, w jaki sposób jądro Linuksa zachowuje stan procesu w momencie rozpoczęcia procedury przerwania (procedura `handle_exception`).⁷

¹<http://pages.cs.wisc.edu/~remzi/OSTEP/intro.pdf>

²<https://wiki.archlinux.org/index.php/xrandr>

³Warto zaznaczyć, że postawienie ujemnych punktów, nie wyklucza wybrania studenta do innego zadania, co w przypadku dalszej niemożności zaprezentowania rozwiązania, może skutkować – w zależności od decyzji prowadzącego – dalszymi skreśleniami i dalszymi punktami ujemnymi.

⁴Jeśli ktoś ma (pomocnych) znajomych, to zadania można prezentować na cudzym komputerze. Tylko że część zadań zapewne będzie wymagała «sudo»...

⁵W tym zadaniu zakładamy nomenklaturę z CSAPP, w ogólności bowiem, nie ma jednego standardu i przykładowo autorzy CVA6 (https://docs.openhwgroup.org/projects/cva6-user-manual/01_cva6_user/Traps_Interrupts_Exceptions.html) piszą, że pułapki to nadzbiór przerw i wyjątków.

⁶<https://elixir.bootlin.com/linux/v6.2.8/source/arch/riscv/kernel/entry.S>

⁷Jeśli wolisz możesz to zadanie wykonać dla pliku `entry.S` z innej architektury np. z x86 lub ARM64.

Zadanie 5. Bazując na formacie ELF (ang. *Executable and Linkable Format*) opisz składowe pliku wykonywalnego. Czym różni się **sekcja** od **segmentu**? Co opisują **nagłówki programu**? Skąd system operacyjny wie, pod jakim adresem ma umieścić segmenty programu i gdzie położona jest pierwsza instrukcja programu?

Wskazówka: Skorzystaj z narzędzia «readelf».

Zadanie 6. Zapoznaj się z rozdziałami 3.4 i A.2 dokumentu [System V Application Binary Interface AMD64 Architecture Processor Supplement⁸](#). Korzystając z niego oraz z CSAPP odpowiedz na następujące pytania:

- W jaki sposób jądro musi przygotować **przestrzeń adresową** procesu? Co musi znajdować się na stosie w momencie wywołania procedury «_start»? Do czego służy *auxiliary vector*? Można go wyświetlić wydając przykładowe polecenie «LD_SHOW_AUXV=1 /bin/true».
- W jaki sposób wywołać funkcję jądra? W których rejestrach należy umieścić argumenty? Gdzie można spodziewać się wyników i jak jądro sygnalizuje niepowodzenie **wywołania systemowego**?

Zadanie 7. Opisz znaczenie słowa kluczowego «volatile» w języku C. Wymień co najmniej dwa scenariusze, w których użycie wskaźników do ulotnej zawartości pamięci jest niezbędne dla poprawności działania programu.

Uwaga: Zadaniem słowa kluczowego «volatile» nie jest wyłączenie optymalizacji!

Zadanie 8. Przypomnij jak wygląda mechanizm **tłumaczenia adresów** bazujący na wielopoziomowej tablicy stron procesorów z rodziny x86-64. Przedstaw algorytm obliczania **adresu fizycznego** na podstawie **adresu wirtualnego** z uwzględnieniem uprawnień dostępu. Jaką rolę w procesie tłumaczenia odgrywa **pamięć TLB**?

Ściągnij ze strony przedmiotu archiwum «so26_lista_0.tar.gz», następnie rozpakuj i skompiluj źródła poleceniem «make».

Zadanie 9. Wyjaśnij, do czego służy narzędzie «ltrace». Uruchom program «1_ls» pod kontrolą «ltrace -S». Na podstawie śladu wykonania programu zidentyfikuj, które z **wywołań systemowych** są używane przez procedury: «opendir», «readdir» i «closedir». Jaka standardowa funkcja biblioteczna jest wywoływana przez «printf»? Jakie wywołanie systemowe przekazuje dane z «printf» do jądra? Dlaczego pojawia się ono z opóźnieniem?

Do czego służy wywołanie systemowe «brk»? Używając debuggera «gdb» i polecenia «catch syscall brk» zidentyfikuj, która funkcja używa «brk».

Wskazówka: Pamiętaj, że «brk» może być wołane więcej niż raz i nie wszystkimi wywołaniami możemy być zainteresowani.

Wskazówka 2: Pamiętaj by korzystać z symboli debugowych dla libc. Jeśli w stosie wywołań funkcji widzisz "??" np. «0x0000000000000002 in ?? ()» to znaczy że nie masz potrzebnych symboli. Najłatwiej je uzyskać z użyciem «debuginfod» (współczesny gdb pyta się na starcie czy ma aktywować «debuginfod»), które pozwala automatycznie pobrać symbole dla danej dystrybucji. Pamiętaj, by wersja libc z której korzystasz odpowiadała wersji dostępnej na serwerach dystrybucji. W skrócie: upewnij się, że masz zainstalowane aktualne pakiety.

Zadanie 10. Pod kontrolą narzędzia «strace» uruchom program «2_cat» korzystający bezpośrednio z wywołań systemowych do interakcji ze **standardowym wejściem i wyjściem**. Pokaż, że program oczekuje na odczyt na deskryptorze pliku 0 i pisze do pliku o deskryptorze 1. Naciśnij kombinację klawiszy «CTRL+D» kończąc wejściowy strumień danych – co zwróciło «read»? Zmodyfikuj program tak, by czytał z pliku podanego w linii poleceń. Co się stanie, jeśli przekażesz **ścieżkę** do katalogu zamiast do pliku zwykłego?

⁸https://www.uclibc.org/docs/psABI-x86_64.pdf